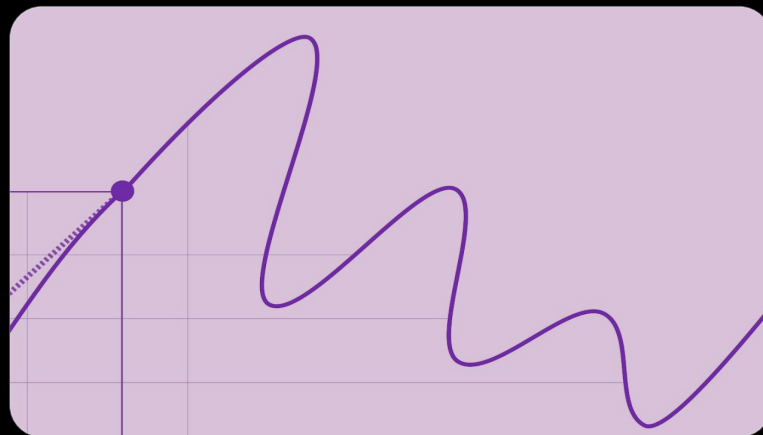
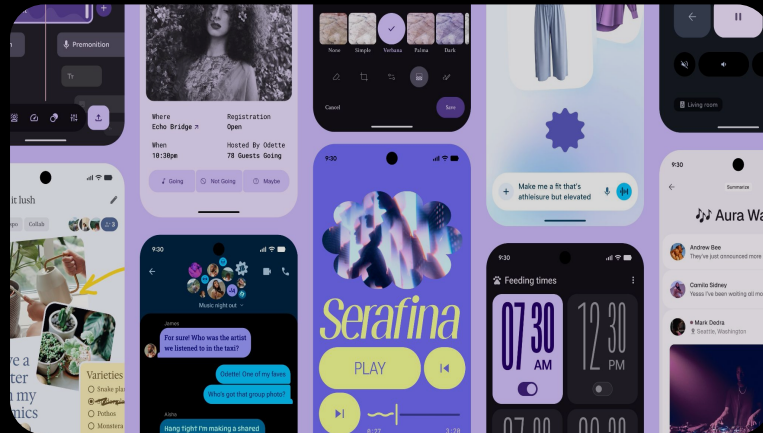


Box, Row e Column

Prof. Dr. Fernando Sambinelli
IFSP Hortolândia

Vamos Começar



Layouts Básicos do Compose

Layouts Fundamentais

- ✱ **Column:** Organiza os elementos verticalmente. Cada filho é empilhado em cima do outro.
- ✱ **Row:** Organiza os elementos horizontalmente. Cada filho é colocado lado a lado.
- ✱ **Box:** Permite empilhar elementos sobrepostos.



Column



Row



Box

Surface

Aparência de Fundo

Surface()

Uma compose que define a aparência de um fundo, permitindo personalizar propriedades como cor, elevação e formato. É usado para criar camadas visuais com os elementos da interface.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            AppBoxRowColumnsTheme {  
                MainScreen(  
                    nome = "Android"  
                )  
            }  
        }  
    }  
}
```

```
@Composable  
fun MainScreen(nome: String) {  
  
    Surface(  
        color = Color.DarkGray,  
        modifier = Modifier.fillMaxSize()  
    )  
    {  
        Text(  
            text = "Hello $nome",  
        )  
    }  
}
```

```
@Preview(showBackground = true)  
@Composable  
fun GreetingPreview() {  
    AppBoxRowColumnsTheme {  
        MainScreen(nome="Android")  
    }  
}
```

GreetingPreview

Hello Android

Wrap Content

Envolver Elementos

wrapContentSize()

Usa-se para envolver conteúdos e personalizar trechos da interface.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppBoxRowColumnsTheme {
                MainScreen(
                    nome = "Android"
                )
            }
        }
    }
}

@Composable
fun MainScreen(nome: String) {

    Surface(
        color = Color.DarkGray,
        modifier = Modifier.fillMaxSize()
    )
    {
        Surface(
            color = Color.Magenta,
            modifier = Modifier.wrapContentSize(
                align = Alignment.TopCenter
            )
        )
        {
            Text(
                text = "Hello $nome",
                style = MaterialTheme.typography.titleMedium
            )
        }
    }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    AppBoxRowColumnsTheme {
        MainScreen(nome="Android")
    }
}
```

GreetingPreview

Hello Android

Múltiplos Composes

Problema no Layout

Problemas

Ao colocar mais de um elemento composable eles ficam sobrepostos.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            AppBoxRowColumnsTheme {  
                MainScreen(  
                    nome = "Android"  
                )  
            }  
        }  
    }  
}
```

```
@Composable  
fun MainScreen(nome: String) {  
  
    Surface(  
        color = Color.DarkGray,  
        modifier = Modifier.fillMaxSize()  
    )  
    {  
        Text(  
            text = "Hello $nome",  
        )  
        Text(  
            text = "Outro texto",  
        )  
    }  
}
```

```
⚡  
@Preview(showBackground = true)  
@Composable  
fun GreetingPreview() {  
    AppBoxRowColumnsTheme {  
        MainScreen(nome="Android")  
    }  
}
```

GreetingPreview

Betho Android

Row

Elementos em Linha

Row()

Organiza os elementos horizontalmente.

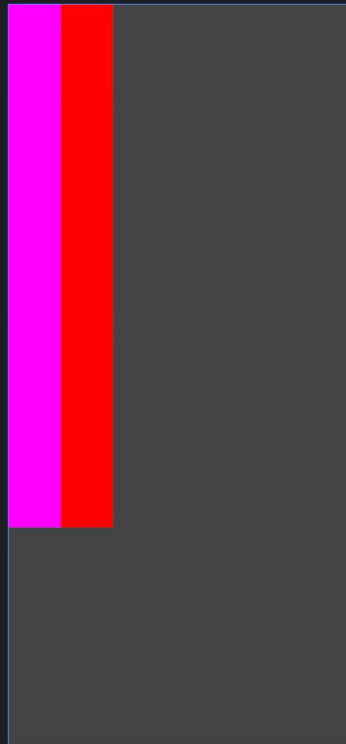
Cada filho é colocado lado a lado.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            AppBoxRowColumnsTheme {  
                MainScreen()  
            }  
        }  
    }  
}
```

```
@Composable  
fun MainScreen() {  
    Surface(  
        color = Color.DarkGray,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        Row {  
            Surface(  
                color = Color.Magenta,  
                modifier = Modifier  
                    .height(600.dp)  
                    .width(60.dp)  
            ) { }  
            Surface(  
                color = Color.Red,  
                modifier = Modifier  
                    .height(600.dp)  
                    .width(60.dp)  
            ) { }  
        }  
    }  
}
```

```
@Preview(showBackground = true)  
@Composable  
fun GreetingPreview() {  
    AppBoxRowColumnsTheme {  
        MainScreen()  
    }  
}
```

GreetingPreview



Reuso de Composes

Otimização

Reuso

É possível aninhar funções composables, gerando reuso de código.

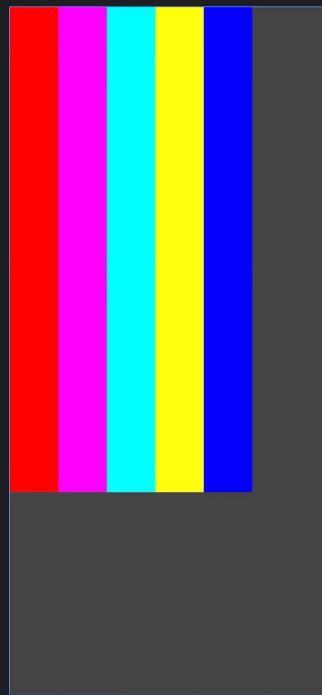
```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppBoxRowColumnsTheme {
                MainScreen()
            }
        }
    }
}

@Composable
fun MainScreen() {
    Surface(
        color = Color.DarkGray,
        modifier = Modifier.fillMaxSize()
    ) {
        Row {
            HorizontalColoredBar(Color.Red)
            HorizontalColoredBar(Color.Magenta)
            HorizontalColoredBar(Color.Cyan)
            HorizontalColoredBar(Color.Yellow)
            HorizontalColoredBar(Color.Blue)
        }
    }
}

@Composable
fun HorizontalColoredBar(color: Color) {
    Surface(
        color = color,
        modifier = Modifier
            .height(60.dp)
            .width(60.dp)
    ) { }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    AppBoxRowColumnsTheme {
        MainScreen()
    }
}
```

GreetingPreview



Alinhamento e Arranjo

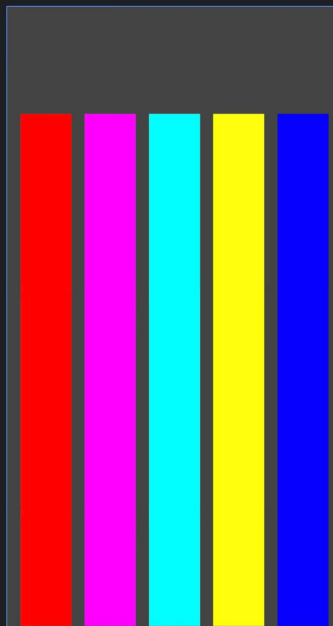
Detalhes sobre Row

Detalhando as Rows

É possível ajustar as Rows usando `horizontalArrangement` e `verticalAlignment`.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            AppBoxRowColumnsTheme {  
                MainScreen()  
            }  
        }  
    }  
}  
  
@Composable  
fun MainScreen() {  
    Surface(  
        color = Color.DarkGray,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        Row(  
            modifier = Modifier.fillMaxSize(),  
            horizontalArrangement = Arrangement.SpaceEvenly,  
            verticalAlignment = Alignment.CenterVertically  
        ) {  
            HorizontalColoredBar(Color.Red)  
            HorizontalColoredBar(Color.Magenta)  
            HorizontalColoredBar(Color.Cyan)  
            HorizontalColoredBar(Color.Yellow)  
            HorizontalColoredBar(Color.Blue)  
        }  
    }  
}  
  
@Composable  
fun HorizontalColoredBar(color: Color) {  
    Surface(  
        color = color,  
        modifier = Modifier  
            .height(600.dp)  
            .width(60.dp)  
    ) { }  
}
```

GreetingPreview



Column

Elementos em Colunas

Column()

Organiza os elementos verticalmente. Cada filho é empilhado em cima do outro.

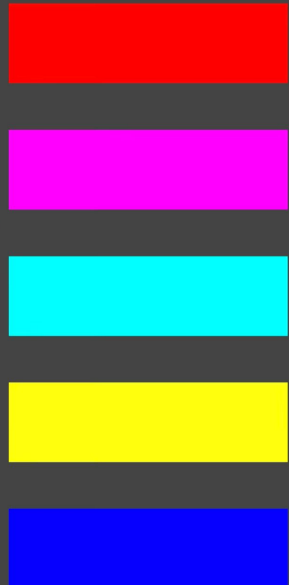
```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppBoxRowColumnsTheme {
                MainScreen()
            }
        }
    }
}

@Composable
fun MainScreen() {
    Surface(
        color = Color.DarkGray,
        modifier = Modifier.fillMaxSize()
    ) {
        Column (
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.SpaceEvenly,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            HorizontalColoredBar(Color.Red)
            HorizontalColoredBar(Color.Magenta)
            HorizontalColoredBar(Color.Cyan)
            HorizontalColoredBar(Color.Yellow)
            HorizontalColoredBar(Color.Blue)
        }
    }
}

@Composable
fun HorizontalColoredBar(color: Color) {
    Surface(
        color = color,
        modifier = Modifier
            .height(100.dp)
            .width(350.dp)
    ) {}
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    AppBoxRowColumnsTheme {
```

GreetingPreview



Combinando Row e Column

Explorando Possibilidades

Rows e Column

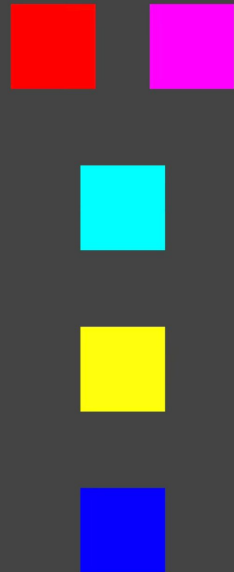
É possível combinar Row e Column e montar layouts bem interessantes.

```
@Composable
fun MainScreen() {
    Surface(
        color = Color.DarkGray,
        modifier = Modifier.fillMaxSize()
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.SpaceEvenly,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceEvenly,
            ) {
                ColoredSquare(Color.Red)
                ColoredSquare(Color.Magenta)
            }
            ColoredSquare(Color.Cyan)
            ColoredSquare(Color.Yellow)
            ColoredSquare(Color.Blue)
        }
    }
}

@Composable
fun ColoredSquare(color: Color) {
    Surface(
        color = color,
        modifier = Modifier
            .height(100.dp)
            .width(100.dp)
    ) {}
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    AppBoxRowColumnsTheme {
        MainScreen()
    }
}
```

GreetingPreview



Box

Elementos em Camadas

Box()

Permite empilhar elementos sobrepostos
(em camadas).

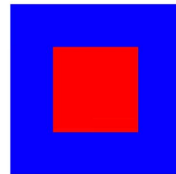
```
@Composable
fun MainScreen(){
    // Box básico - permite empilhar elementos
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.White)
    ) {
        // Elemento no fundo (primeira camada)
        Box(
            modifier = Modifier
                .size(200.dp)
                .background(Color.Blue)
                .align(Alignment.Center)
        )

        // Elemento sobreposto (segunda camada)
        Box(
            modifier = Modifier
                .size(100.dp)
                .background(Color.Red)
                .align(Alignment.Center)
        )

        // Texto sobreposto (terceira camada)
        Text(
            text = "Exemplo de Box",
            modifier = Modifier
                .align(Alignment.BottomCenter)
                .padding(16.dp)
        )
    }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    AppBoxRowColumnsTheme {
        MainScreen()
    }
}
```

GreetingPreview



Exemplo de Box



Fernando Sambinelli

Professor e Doutor em Tecnologia
sambinelli@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo

